4-3 程序的机器级代码表示

- 1. 常用汇编语句
  - 1.1 数据传送指令
    - mov 指令
      - **作用**：将第二个操作数复制到第一个操作数。
      - **格式例子**：`mov eax, ebx`。
      - **内存写入例子**：`mov byte ptr [var], 5`。
      - **限制**：两个操作数不可同时为内存。
      - **理解**：`mov 目的, 源`，把源操作数复制到目的操作数。
    - push 指令
      - **作用**：将操作数压入栈中。
      - **常见用途**：函数调用、现场保护。
      - **栈顶寄存器**：ESP 指向栈顶。
      - **入栈过程**：入栈前先 `ESP = ESP - 4`，因为栈向低地址方向增长；再把操作数写入 ESP 所指地址。
    - pop 指令
      - **作用**：从栈中弹出数据。
      - **出栈过程**：先读出 ESP 所指地址的内容，再 `ESP = ESP + 4`。
  - 1.2 算术和逻辑运算指令
    - add/sub 指令
      - **add**：将两个操作数相加。
      - **sub**：用第一个操作数减去第二个操作数。
      - **结果保存**：结果均保存在第一个操作数中。
    - inc/dec 指令
      - **inc**：对操作数自增 1。
      - **dec**：对操作数自减 1。
    - imul 指令
      - **作用**：带符号乘法。
      - **双操作数格式**：第一操作数 x 第二操作数，结果存入第一个操作数；第一个操作数必须是寄存器。
      - **三操作数格式**：第二操作数 x 第三操作数，结果存入第一个操作数；第一个操作数必须是寄存器。
    - idiv 指令
      - **作用**：带符号除法。
      - **操作数特点**：指令中只指定除数。
      - **被除数**：由 `edx:eax` 组成 64 位有符号数。
      - **执行结果**：商存入 `eax`，余数存入 `edx`。
    - and/or/xor 指令
      - **and**：按位与。
      - **or**：按位或。
      - **xor**：按位异或。
      - **结果保存**：结果存入第一个操作数。
      - **常见用法**：`xor edx, edx` 可把 `edx` 清零。
    - not 指令
      - **作用**：按位取反。
    - neg 指令
      - **作用**：取负。
    - shl/shr 指令
      - **shl**：逻辑左移。
      - **shr**：逻辑右移。
      - **操作数含义**：第一个操作数是被移位数，第二个操作数是移位位数。
  - 1.3 控制流指令
    - jmp 指令
      - **作用**：无条件转移。
      - **执行效果**：跳转到 label 标签所指示的地址继续执行。
    - jcondition 指令
      - **作用**：条件转移。
      - **判断依据**：根据程序状态字寄存器 PSW 中的状态标志决定是否转移。
      - **je**：相等时转移。
      - **jz**：结果为 0 时转移。
      - **jne**：不相等时转移。
      - **jg**：大于时转移。
      - **jge**：大于等于时转移。
      - **jl**：小于时转移。
      - **jle**：小于等于时转移。
    - cmp/test 指令
      - **cmp**：执行减法运算但不保存结果，只根据结果设置标志位。
      - **test**：执行按位与运算但不保存结果，只更新标志位。
      - **常见搭配**：通常与条件转移指令配合使用。
- 2. 选择语句的机器代码表示
  - C 语言逻辑
    - **函数含义**：比较两个指针 `p1` 和 `p2`。
    - **条件为真**：若 `p1 > p2`，返回 `*p2`。
    - **条件为假**：否则返回 `*p1`。
  - 汇编执行思路
    - **加载参数**：从栈帧中取出 `p1` 和 `p2`，分别放入寄存器。
    - **比较条件**：使用 `cmp eax, edx` 比较 `p1` 和 `p2`。
    - **条件跳转**：使用 `jbe .L1`，若 `p1 <= p2`，跳转到 else 分支。
    - **then 分支**：若 `p1 > p2`，取 `*p2` 作为返回值。
    - **跳过 else**：then 分支执行完后用 `jmp .L2` 跳过 else 分支。
    - **else 分支**：取 `*p1` 作为返回值。
    - **返回位置**：返回值通常放在 `eax` 中。
  - 关键结构
    - **cmp + 条件跳转**：形成 if/else 的分支判断。
    - **label 标签**：标记分支入口和汇合位置。
    - **jmp**：用于跳过不该执行的分支。
- 3. 循环语句的机器代码表示
  - C 语言逻辑
    - **函数含义**：计算 `1 + 2 + ... + n`。
    - **初始化**：`result = 0`，`i = 1`。
    - **循环条件**：`i <= n`。
    - **循环体**：`result += i`。
    - **更新表达式**：`i++`。
  - 汇编执行思路
    - **加载 n**：把参数 `n` 放入 `ecx`。
    - **初始化 result**：`mov eax, 0`，返回值/累加值放在 `eax`。
    - **初始化 i**：`mov edx, 1`，循环变量放在 `edx`。
    - **第一次判断**：`cmp edx, ecx` 比较 `i` 与 `n`。
    - **跳出循环**：若 `i > n`，使用 `jg .L2` 跳到循环结束。
    - **循环体入口**：`.L1`。
    - **累加**：`add eax, edx`，即 `result += i`。
    - **自增**：`add edx, 1`，即 `i++`。
    - **再次比较**：继续比较 `i` 与 `n`。
    - **回到循环体**：若 `i <= n`，使用 `jle .L1`。
    - **循环结束**：`.L2`。
  - 关键结构
    - **初始化代码**：进入循环前设置变量初值。
    - **条件判断**：使用 `cmp` 设置标志位。
    - **条件跳转**：决定进入循环体还是退出。
    - **循环回跳**：循环体末尾跳回 `.L1`。
- 4. 过程调用的机器代码表示
  - 过程调用步骤
    - **1. 放置参数**：调用者 P 将入口参数放到被调用者 Q 能访问的位置。
    - **2. call 指令**：P 执行 `call Q`，把返回地址压入栈中，并跳转到 Q。
    - **3. 建立栈帧**：Q 建立自己的栈帧，为局部变量分配空间，必要时保存寄存器。
    - **4. 执行主体**：Q 执行函数体代码。
    - **5. 准备返回**：Q 将返回结果放到约定位置，释放局部变量空间，恢复保存过的寄存器。
    - **6. ret 指令**：Q 执行 `ret`，从栈顶弹出返回地址，并回到 P 继续执行。
  - 调用者保存寄存器
    - **寄存器**：`EAX`、`ECX`、`EDX`。
    - **保护责任**：如果调用者 P 调用 Q 后还要继续使用这些寄存器中的旧值，P 必须在调用前自己保存。
    - **常见保存位置**：栈。
  - 被调用者保存寄存器
    - **寄存器**：`EBX`、`ESI`、`EDI`。
    - **保护责任**：如果 Q 想使用这些寄存器，必须先保存，返回前恢复。
    - **调用者假设**：P 默认相信这些寄存器在调用前后不会变化。
  - 栈帧
    - **定义**：每次过程/函数被调用时，系统在栈中为其分配的一块独立存储区域。
    - **保存内容**：局部变量、返回地址、参数、旧 EBP 等信息。
    - **ESP**：栈顶指针，永远指向当前栈顶。
    - **EBP**：栈帧指针，指向当前函数栈帧的基准位置。
    - **栈增长方向**：栈通常向低地址方向增长。
  - caller 调用 add 的栈帧示例
    - **保存旧 EBP**：`push ebp` 保存调用者 P 的 EBP。
    - **建立新栈帧**：`mov ebp, esp`，让 EBP 指向当前栈顶。
    - **分配局部空间**：`sub esp, 24`，为局部变量和参数区分配空间。
    - **保存局部变量**：`temp1 = 125`、`temp2 = 80` 存入栈帧。
    - **准备实参**：把 `temp1`、`temp2` 放入参数区。
    - **调用 add**：`call add`，返回值保存在 `eax`。
    - **保存返回值**：`sum = eax`。
    - **准备返回**：把 `sum` 放入 `eax`。
    - **恢复栈帧**：`leave` 等价于 `mov esp, ebp; pop ebp`。
    - **返回调用者**：`ret`。
  - 栈帧访问习惯
    - **正偏移**：常用于访问参数和返回地址附近内容。
    - **负偏移**：常用于访问局部变量。
    - **返回值位置**：x86 常见约定中，整数返回值放在 `eax`。

后续关注公众号[研大]
永久微信1931637888

by joshuji 由 幕布 发布